

In internet security, cross-site (XS) leaks are a class of attacks used to access a user's sensitive information on another website. Cross-site leaks allow an attacker to access a user's interactions with other websites. This can contain sensitive information. Web browsers normally stop other websites from seeing this information. This is enforced through a set of rules called the same-origin policy. Attackers can sometimes get around these rules, using a "cross-site leak". Attacks using a cross-site leak are often initiated by enticing users to visit the attacker's website. Upon visiting, the attacker uses malicious code on their website to interact with another website. This can be used by an attacker to learn about the user's previous actions on the other website. The information from this attack can uniquely identify the user to the attacker.

These attacks have been documented since 2000. One of the first research papers on the topic was published by researchers at Purdue University. The paper described an attack where the web cache was exploited to gather information about a website. Since then, cross-site leaks have become increasingly sophisticated. Researchers have found newer leaks targeting various web browser components. While the efficacy of some of these techniques varies, newer techniques are continually being discovered. Some older methods are blocked through updates to browsers. The introduction and removal of features on the Internet also lead to some attacks being rendered ineffective.

Cross-site leaks are a diverse form of attack, and there is no consistent classification of such attacks. Multiple sources classify cross-site leaks by the technique used to leak information. Among the well-known cross-site leaks are timing attacks, which depend on timing events within the web browser. For example, cache-timing attacks rely on the web cache to unveil information. Error events constitute another category, using the presence or absence of events to disclose data. Since 2023, newer attacks that use operating systems and web browser limits to leak information have also been found.

Before 2017, defending against cross-site leaks was considered to be difficult. This was

because many of the information leakage issues exploited by cross-site leak attacks were inherent to the way websites worked. Most defences against this class of attacks have been introduced after 2017 in the form of extensions to the hypertext transfer protocol (HTTP). These extensions allow websites to instruct the browser to disallow or annotate certain kinds of stateful requests coming from other websites. One of the most successful approaches browsers have implemented is SameSite cookies. SameSite cookies allow websites to set a directive that prevents other websites from accessing and sending sensitive cookies. Another defence involves using HTTP headers to restrict which websites can embed a particular site. Cache partitioning also serves as a defence against cross-site leaks, preventing other websites from using the web cache to exfiltrate data.

Background

Web applications (web apps) have two primary components: a web browser and one or more web servers. The browser typically interacts with the servers via hyper text transfer protocol (HTTP) and WebSocket connections to deliver a web app. To make the web app interactive, the browser also renders HTML and CSS, and executes JavaScript code provided by the web app. These elements allow the web app to react to user inputs and run client-side logic. Often, users interact with the web app over long periods, making multiple requests to the server. To keep track of such requests, web apps often use a persistent identifier tied to a specific user through their current session or user account. This identifier can include details like age or access level, which reflect the user's history with the web app. If revealed to other websites, these identifiable attributes might deanonymize the user.

Ideally, each web app should operate independently without interfering with others. However, due to various design choices made during the early years of the web, web apps can regularly interact with each other. To prevent the abuse of this behavior, web browsers enforce a set of rules called the same-origin policy that limits direct interactions between web applications

from different sources. Despite these restrictions, web apps often need to load content from external sources, such as instructions for displaying elements on a page, design layouts, and videos or images. These types of interactions, called cross-origin requests, are exceptions to the same-origin policy. They are governed by a set of strict rules known as the cross-origin resource sharing (CORS) framework. CORS ensures that such interactions occur under controlled conditions by preventing unauthorized access to data that a web app is not allowed to see. This is achieved by requiring explicit permission before other websites can access the contents of these requests.

Cross-site leaks allow attackers to circumvent the restrictions imposed by the same-origin policy and the CORS framework. They leverage information-leakage issues (side channels) that have historically been present in browsers. Using these side channels, an attacker can execute code that can infer details about data that the same origin policy would have shielded. This data can then be used to reveal information about a user's previous interactions with a web app.

Mechanism

To carry out a cross-site leak attack, an attacker must first study how a website interacts with users. They need to identify a specific URL that produces different Hyper Text Transfer Protocol (HTTP) responses based on the user's past actions on the site. For instance, if the attacker is trying to attack Gmail, they could try to find a search URL that returns a different HTTP response based on how many search results are found for a specific search term in a user's emails. Once an attacker finds a specific URL, they can then host a website and phish or otherwise lure unsuspecting users to the website. Once the victim is on the attacker's website, the attacker can use various embedding techniques to initiate cross-origin HTTP requests to the URL identified by the attacker. However, since the attacker is on a different website, the same-origin policy imposed by the web browser will prevent the attacker from directly reading

any part of the response sent by the vulnerable website.

To circumvent this security barrier, the attacker can use browser-leak methods, to distinguish subtle differences between different responses. Browser leak methods are JavaScript, CSS or HTML snippets that leverage long-standing information leakage issues (side channels) in the web browser to reveal specific characteristics about a HTTP response. In the case of Gmail, the attacker could use JavaScript to time how long the browser took to parse the HTTP response returned by the search result. If the time taken to parse the response returned by the endpoint was low, the attacker could infer that there were no search results for their query. Alternatively, if the site took longer, the attacker could infer that multiple search results were returned. The attacker can subsequently use the information gained through these information leakages to exfiltrate sensitive information, which can be used to track and deanonymize the victim. In the case of Gmail, the attacker could make a request to the search endpoint with a query and subsequently measure the time the query took to figure out whether or not the user had any emails containing a specific query string. If a response takes very little time to be processed, the attacker can assume that no search results were returned. Conversely, if a response takes a large amount of time to be processed, the attacker infer that a lot of search results were returned. By making multiple requests, an attacker could gain significant insight into the current state of the victim application, potentially revealing private information of a user, helping launch sophisticated spamming and phishing attacks.

History

Cross-site leaks have been known about since 2000; research papers dating from that year from Purdue University describe a theoretical attack that uses the HTTP cache to compromise the privacy of a user's browsing habits. In 2007, Andrew Bortz and Dan Boneh from Stanford University published a white paper detailing an attack that made use of timing information to determine the size of cross-site responses. In 2015, researchers from Bar-Ilan University

described a cross-site search attack that used similar leaking methods. The attack employed a technique in which the input was crafted to grow the size of the responses, leading to a proportional growth in the time taken to generate the responses, thus increasing the attack's accuracy.

Independent security researchers have published blog posts describing cross-site leak attacks against real-world applications. In 2009, Chris Evans described an attack against Yahoo! Mail via which a malicious site could search a user's inbox for sensitive information. In 2018, Luan Herrera found a cross-site leak vulnerability in Google's Monorail bug tracker, which is used by projects like Chromium, Angle, and Skia Graphics Engine. This exploit allowed Herrera to exfiltrate data about sensitive security issues by abusing the search endpoint of the bug tracker. In 2019, Terjanq, a Polish security researcher, published a blog post describing a cross-site search attack that allowed them to exfiltrate sensitive user information across high-profile Google products.

As part of its increased focus on dealing with security issues that depend on misusing long-standing web-platform features, Google launched XSLeaks Wiki in 2020. The initiative aimed to create an open-knowledge database about web-platform features that were being misused and analysing and compiling information about cross-site leak attacks.

Since 2020, there has been some interest among the academic security community in standardizing the classification of these attacks. In 2020, Sudhodanan et al. were among the first to systematically summarize previous work in cross-site leaks, and developed a tool called BASTA-COSI that could be used to detect leaky URLs. In 2021, Knittel et al. proposed a new formal model to evaluate and characterize cross-site leaks, allowing the researchers to find new leaks affecting several browsers. In 2022, Van Goethem et al. evaluated currently available defences against these attacks and extended the existing model to consider the state of browser components as part of the model. In 2023, a paper published by Rautenstrauch et al. systemizing previous research into cross-site leaks was awarded the

Distinguished Paper Award at the IEEE Symposium on Security and Privacy.

Threat model

The threat model of a cross-site leak relies on the attacker being able to direct the victim to a malicious website that is at least partially under the attacker's control. The attacker can accomplish this by compromising a web page, by phishing the user to a web page and loading arbitrary code, or by using a malicious advertisement on an otherwise-safe web page.

Cross site leak attacks require that the attacker identify at least one state-dependent URL in the victim app for use in the attack app. Depending on the victim app's state, this URL must provide at least two responses. A URL can be crafted, for example, by linking to content that is only accessible to the user if they are logged into the target website. Including this state-dependent URL in the malicious application will initiate a cross-origin request to the target app. Because the request is a cross-origin request, the same-origin policy prevents the attacker from reading the contents of the response. Using a browser-leak method, however, the attacker can query specific identifiable characteristics of the response, such as the HTTP status code. This allows the attacker to distinguish between responses and gain insight into the victim app's state.

While every method of initiating a cross-origin request to a URL in a web page can be combined with every browser-leak method, this does not work in practice because dependencies exist between different inclusion methods and browser leaks. Some browser-leak methods require specific inclusion techniques to succeed. For example, if the browser-leak method relies on checking CSS attributes such as the width and height of an element, the inclusion technique must use an HTML element with a width and height property, such as an image element, that changes when a cross-origin request returns an invalid or a differently sized image.

Types

Cross-site leaks comprise a highly varied range of attacks for which there is no established, uniform classification. However, multiple sources typically categorized these attacks by the leaking techniques used during an attack. As of 2021, researchers have identified over 38 leak techniques that target components of the browser. New techniques are typically discovered due to changes in web platform APIs, which are JavaScript interfaces that allow websites to query the browser for specific information. Although the majority of these techniques involve directly detecting state changes in the victim web app, some attacks also exploit alterations in shared components within the browser to indirectly glean information about the victim web app.

Timing attacks

Timing attacks rely on the ability to time specific events across multiple responses. These were discovered by researchers at Stanford University in 2007, making them one of the oldest-known types of cross-site leak attacks.

While initially used only to differentiate between the time it took for a HTTP request to resolve a response, research performed after 2007 has demonstrated the use of this leak technique to detect other differences across web-app states. In 2017, Vila et al. showed timing attacks could infer cross-origin execution times across embedded contexts. This was made possible by a lack of site isolation features in contemporaneous browsers, which allowed an attacking website to slow down and amplify timing differences caused by differences in the amount of JavaScript being executed when events were sent to a victim web app.

In 2021, Knittel et al. showed the Performance API could leak the presence or absence of redirects in responses. This was possible due to a bug in the Performance API that allowed the amount of time shown to the user to be negative when a redirect occurred. Google Chrome subsequently fixed this bug. In 2023, Snyder et al. showed timing attacks could be used to

perform pool-party attacks in which websites could block shared resources by exhausting their global quota. By making the victim web app execute JavaScript that used these shared resources and then timing how long these executions took, the researchers were able to reveal information about the state of a web app.

Error events

Error events is a leak technique that allows an attacker to distinguish between multiple responses by registering error-event handlers and listening for events through them. Due to their versatility and ability to leak a wide range of information, error events are considered a classic cross-site leak vector.

One of the most-common use cases for error events in cross-site leak attacks is determining HTTP responses by attaching the event handlers `onload` and `onerror` event handlers to a HTML element and waiting for specific error events to occur. A lack of error events indicates no HTTP errors occurred. In contrast, if the handler `onerror` is triggered with a specific error event, the attacker can use that information to distinguish between HTTP content types, status codes and media-type errors. In 2019, researchers from TU Darmstadt showed this technique could be used to perform a targeted deanonymization attack against users of popular web services such as Dropbox, Google Docs, and GitHub that allow users to share arbitrary content with each other.

Since 2019, the capabilities of error events have been expanded. In 2020, Janc et al. showed by setting the `redirect` mode for a `fetch` request to `manual`, a website could leak information about whether a specific URL is a redirect. Around the same time, Jon Masas and Luan Herrera showed by abusing URL-related limits, an attacker could trigger error events that could be used to leak redirect information about URLs. In 2021, Knittel et al. showed error events that are generated by a subresource integrity check, a mechanism that is used to confirm a sub-resource a website loads has not been changed or compromised, could also be used to

guess the raw content of an HTTP response and to leak the content-length of the response.

Cache-timing attacks

Cache-timing attacks rely on the ability to infer hits and misses in shared caches on the web platform. One of the first instances of a cache-timing attack involved the making of a cross-origin request to a page and then probing for the existence of the resources loaded by the request in the shared HTTP and the DNS cache. The paper describing the attack was written by researchers at Purdue University in 2000, and describes the attack's ability to leak a large portion of a user's browsing history by selectively checking if resources that are unique to a web page have been loaded.

This attack has become increasingly sophisticated, allowing the leakage of other types of information. In 2014, Jia et al. showed this attack could geo-locate a person by measuring the time it takes for the localized domain of a group of multinational websites to load. In 2015, Van Goethem et al. showed using the then-newly introduced application cache, a website could instruct the browser to disregard and override any caching directive the victim website sends. The paper also demonstrated a website could gain information about the size of the cached response by timing the cache access.

Global limits

Global limits, which are also known as pool-party attacks, do not directly rely on the state of the victim web app. This cross-site leak was first discovered by Knittel et al. in 2020 and then expanded by Snyder et al. in 2023. The attack abuses global operating systems or hardware limitations to starve shared resources. Global limits that could be abused include the number of raw socket connections that can be registered and the number of service workers that can be registered. An attacker can infer the state of the victim website by performing an activity that triggers these global limits and comparing any differences in browser behaviour when

the same activity is performed without the victim website being loaded. Since these types of attacks typically also require timing side channels, they are also considered timing attacks.

Other techniques

In 2019, Gareth Heyes discovered that by setting the URL hash of a website to a specific value and subsequently detecting whether a loss of focus on the current web page occurred, an attacker could determine the presence and position of elements on a victim website. In 2020, Knittel et al. showed an attacker could leak whether or not a Cross-Origin-Opener-Policy header was set by obtaining a reference to the window object of a victim website by framing the website or by creating a popup of the victim website. Using the same technique of obtaining window references, an attacker could also count the number of frames a victim website had through the `window.length` property.

While newer techniques continue to be found, older techniques for performing cross-site leaks have become obsolete due to changes in the World Wide Web Consortium (W3C) specifications and updates to browsers. In December 2020, Apple updated its browser Safari's Intelligent Tracking Prevention (ITP) mechanism, rendering a variety of cross-site leak techniques researchers at Google had discovered ineffective. Similarly, the widespread introduction of cache partitioning in all major browsers in 2020 has reduced the potency of cache-timing attacks.

Example

The example of a Python-based web application with a search endpoint interface implemented using the following Jinja template demonstrates a common scenario of how a cross-site leak attack could occur.

This code is a template for displaying search results on a webpage. It loops through a

collection of results provided by a HTTP server backend and displays each result along with its description inside a structured div element alongside an icon loaded from a different website. The underlying application authenticates the user based on cookies that are attached to the request and performs a textual search of the user's private information using a string provided in a GET parameter. For every result returned, an icon that is loaded from a Content Delivery Network (CDN) is shown alongside the result.

This simple functionality is vulnerable to a cross-leak attack, as shown by the following JavaScript snippet.

This JavaScript snippet, which can be embedded in an attacker-controlled web app, loads the victim web app inside an iframe, waits for the document to load and subsequently requests the icon from the CDN. The attacker can determine whether the icon was cached by timing its return. Because the icon will only be cached if and only if the victim app returns at least one result, the attacker can determine whether the victim app returned any results for the given query.

Defences

Before 2017, websites could defend against cross-site leaks by ensuring the same response was returned for all application states, thwarting the attacker's ability to differentiate the requests. This approach was infeasible for any non-trivial website. The second approach was to create session-specific URLs that would not work outside a user's session. This approach limited link sharing, and was impractical.

Most modern defences are extensions to the HTTP protocol that either prevent state changes, make cross-origin requests stateless, or completely isolate shared resources across multiple origins.

Isolating shared resources

One of the earliest methods of performing cross-site leaks was using the HTTP cache, an approach that relied on querying the browser cache for unique resources a victim's website might have loaded. By measuring the time it took for a cross-origin request to resolve an attacking website, one could determine whether the resource was cached and, if so, the state of the victim app. As of October 2020, most browsers have implemented HTTP cache partitioning, drastically reducing the effectiveness of this approach. HTTP cache partitioning works by multi-keying each cached request depending on which website requested the resource. This means if a website loads and caches a resource, the cached request is linked to a unique key generated from the resource's URL and that of the requesting website. If another website attempts to access the same resource, the request will be treated as a cache miss unless that website has previously cached an identical request. This prevents an attacking website from deducing whether a resource has been cached by a victim website.

Another, more developer-oriented feature that allows the isolation of execution contexts includes the Cross-Origin-Opener-Policy (COOP) header, which was originally added to address Spectre issues in the browser. It has proved useful for preventing cross-site leaks because if the header is set with a same-origin directive as part of the response, the browser will disallow cross-origin websites from being able to hold a reference to the defending website when it is opened from a third-party page.

As part of an effort to mitigate cross-site leaks, the developers of all major browsers have implemented storage partitioning, allowing all shared resources used by each website to be multi-keyed, dramatically reducing the number of inclusion techniques that can infer the states of a web app.

Preventing state changes

Cross-site leak attacks rely on a malicious web page's ability to receive cross-origin responses

from the victim application. By preventing the malicious application from being able to receive cross-origin responses, the user is no longer in danger of having state changes leaked. This approach is seen in defences such as the deprecated X-Frame-Options header and the newer frame-ancestors directive in Content-Security Policy headers, which allow the victim application to specify which websites can include it as an embedded frame. If the victim app disallows the embedding of the website in untrusted contexts, the malicious app can no longer observe the response to cross-origin requests made to the victim app using the embedded frame technique.

A similar approach is taken by the Cross-Origin Resource Blocking (CORB) mechanism and the Cross-Origin-Resource-Policy (CORS) header, which allows a cross-origin request to succeed but blocks the loading of the content in third-party websites if there is a mismatch between the content type that was expected and that which was received. This feature was originally introduced as part of a series of mitigations against the Spectre vulnerability but it has proved useful in preventing cross-origin leaks because it blocks the malicious web page from receiving the response and thus inferring state changes.

Making cross-origin requests stateless

One of the most-effective approaches to mitigating cross-site leaks has been the use of the SameSite parameter in cookies. Once set to Lax or Strict, this parameter prevents the browser from sending cookies in most third-party requests, effectively making the request stateless. Adoption of Same-Site cookies, however, has been slow because it requires changes in the way many specialized web servers, such as authentication providers, operate. In 2020, the makers of the Chrome browser announced they would be turning on SameSite=Lax as the default state for cookies across all platforms. Despite this, there are still cases in which SameSite=Lax cookies are not respected, such as Chrome's LAX+POST mitigation, which allows a cross-origin site to use a SameSite=Lax cookie in a request if and only if the request is sent

while navigating the page and it occurs within two minutes of the cookie being set. This has led to bypasses and workarounds against the SameSite=Lax limitation that still allow cross-site leaks to occur.

Fetch metadata headers, which include the Sec-Fetch-Site, Sec-Fetch-Mode, Sec-Fetch-User and Sec-Fetch-Dest header, which provide information about the domain that initiated the request, details about the request's initiation, and the destination of the request respectively to the defending web server, have also been used to mitigate cross-site leak attacks. These headers allow the web server to distinguish between legitimate third-party, same-site requests and harmful cross-origin requests. By discriminating between these requests, the server can send a stateless response to malicious third-party requests and a stateful response to routine same-site requests. To prevent the abusive use of these headers, a web app is not allowed to set these headers, which must only be set by the browser.

See also

[Cross origin resource sharing](#)

[Same origin policy](#)

[Cross-site scripting](#)

[Cross-site request forgery](#)

References

Notes

Citations

Sources

Further reading

External links

"XSLeaks Wiki - Introduction". xsleaks.dev.

